

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 3

Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026– 27

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2. Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

3. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

N Input Size

F Filter Size

P Padding

S Stride

3.1 Common Convolution Kernels

A **kernel** (or **filter**) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects **vertical edges** by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects **horizontal edges** by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:

- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:

- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:

- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:

- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

4. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output

First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

5. Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,
Window 1

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

Window 2

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

Window 3

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

Window 4

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

6. Numerical Example 3: Parameter Calculation

Suppose

Input Image

$$32 \times 32 \times 3$$

Convolution Layer

- Filters = 16
- Kernel = 3×3

Parameters

$$(3 \times 3 \times 3 + 1) \times 16$$

$$= (27 + 1) \times 16$$

$$= 448$$

Therefore,
Total trainable parameters = 448.

7. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

8. Experimental Procedure

Task 1

Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

Task 2

Implement a convolution layer.

Compare

- Kernel = 3×3
- Kernel = 5×5
- Kernel = 7×7

Record feature map sizes.

Task 3

Study hyperparameters.

Compare

- Stride = 1
- Stride = 2
- Padding = Same
- Padding = Valid

Compute output dimensions.

Task 4

Visualize feature maps after the first convolution layer.

Display at least eight feature maps.

Task 5

Compare

- Max Pooling
- Average Pooling

Compare output size and accuracy.

Task 6

Construct the following CNN.

$Input \rightarrow Conv \rightarrow ReLU \rightarrow MaxPool \rightarrow Conv \rightarrow ReLU \rightarrow MaxPool \rightarrow Flatten \rightarrow Dense \rightarrow Softmax$

Train for

- Optimizer : Adam
- Epochs : 20
- Batch Size : 32

Task 7

Evaluate

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

9. Mandatory Plots and Inferences

9.1 Sample Images

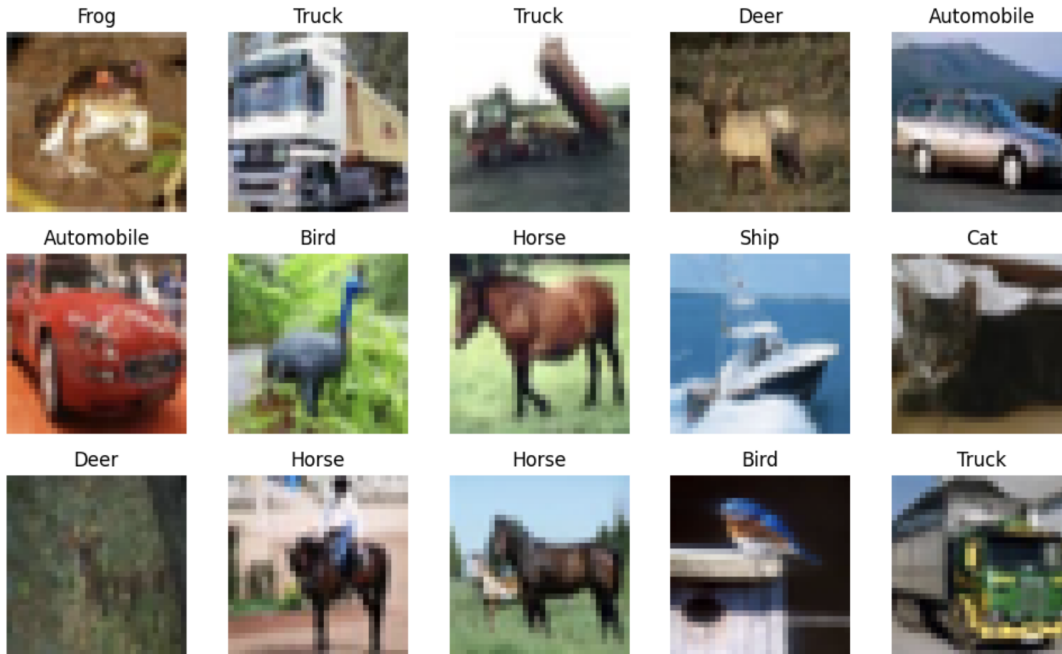


Figure 1: Sample Images

Inference: The sample images of the 10 CIFAR-10 classes are displayed. These are RGB images, but as the images are of low resolution, some objects are not very clear, which can make classification more difficult.

9.2 Class Distribution

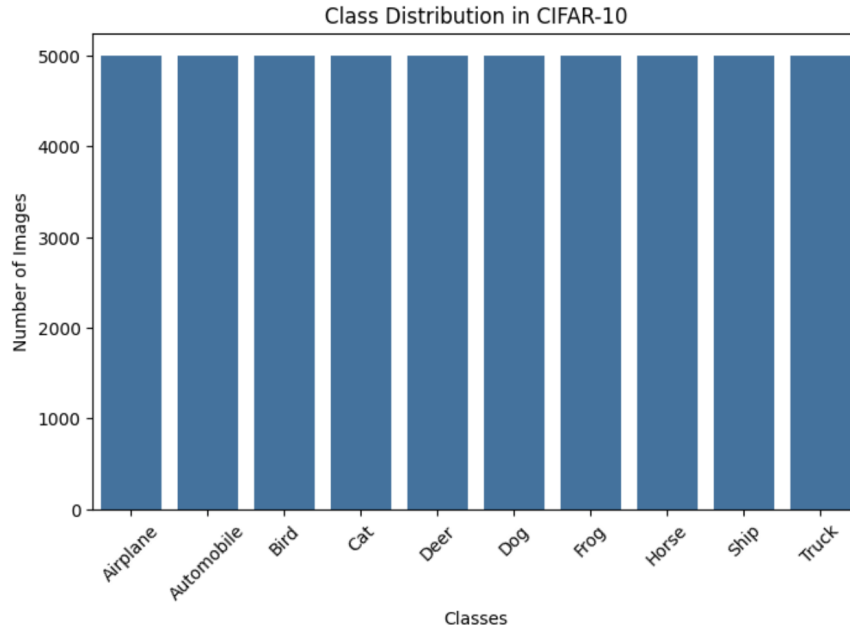


Figure 2: Class Distribution

Inference: The CIFAR-10 training dataset contains 5,000 images for each of the 10 classes. The graph shows that all the classes have same number of training images. Therefore, there is no class imbalance in the training dataset. It is a balanced dataset.

9.3 Training and Validation Accuracy

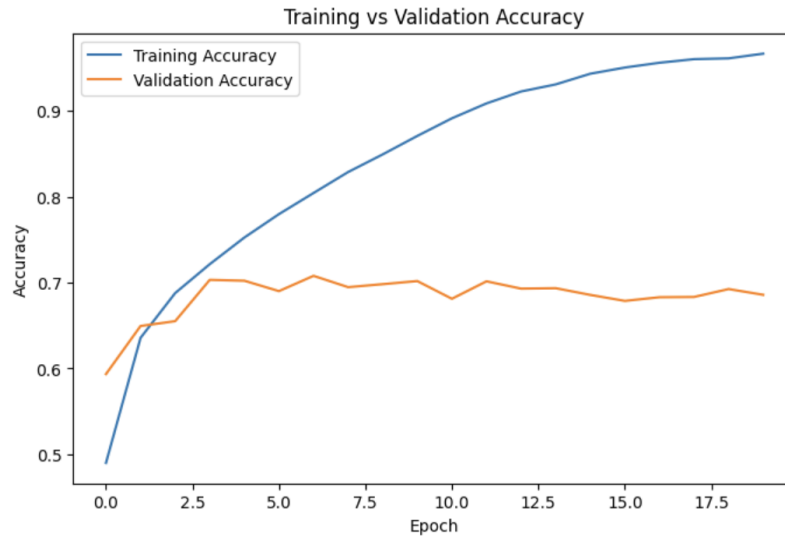


Figure 3: Training and Validation Accuracy

Inference: The training and validation accuracy both are increasing in the initial epochs, showing that the model is learning from the CIFAR-10 images. The validation accuracy reaches about 0.7032 around

epoch 4, but after that it stays around 0.68–0.71 while training accuracy keeps increasing and reaches 0.9666 by epoch 20. This increasing gap shows that the model is fitting the training data more, but is not improving much on the validation data. So, the later epochs show signs of overfitting. Hence, using the final epoch weights would not be the best choice for evaluating the model.

9.4 Training and Validation Loss

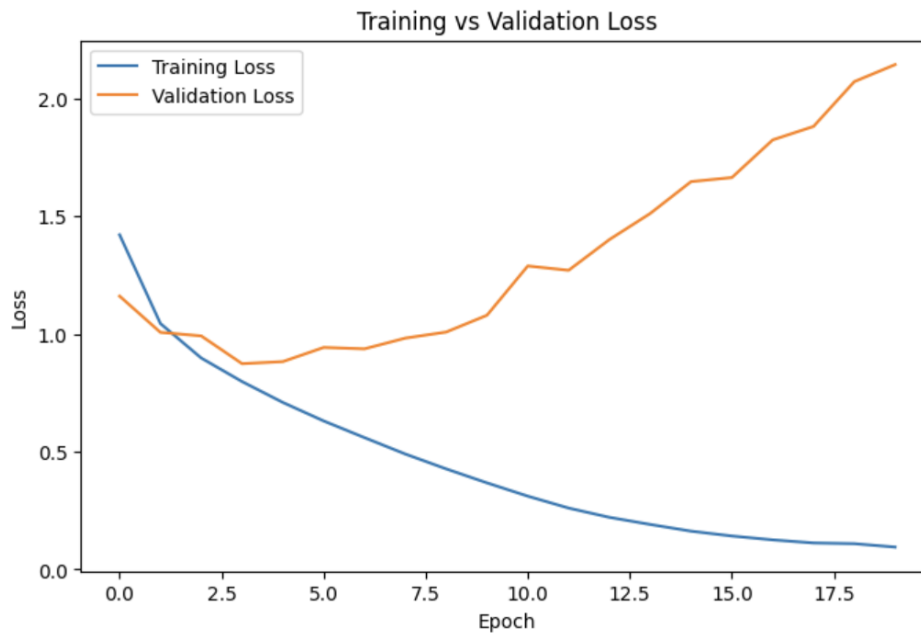


Figure 4: Training and Validation Loss

Inference: The training loss keeps decreasing throughout the 20 epochs and reaches about 0.0950 at epoch 20. Whereas, the validation loss decreases initially and reaches its lowest value of 0.87342 at epoch 4, but then it starts increasing and reaches 2.1429 at epoch 20. This difference between the training and validation loss shows that the model is fitting the training data more, while its performance on the validation data is getting worse. Hence, the weights from the epoch with the best validation loss were saved and used for the final evaluation instead of the final epoch weights.

9.5 Feature Maps

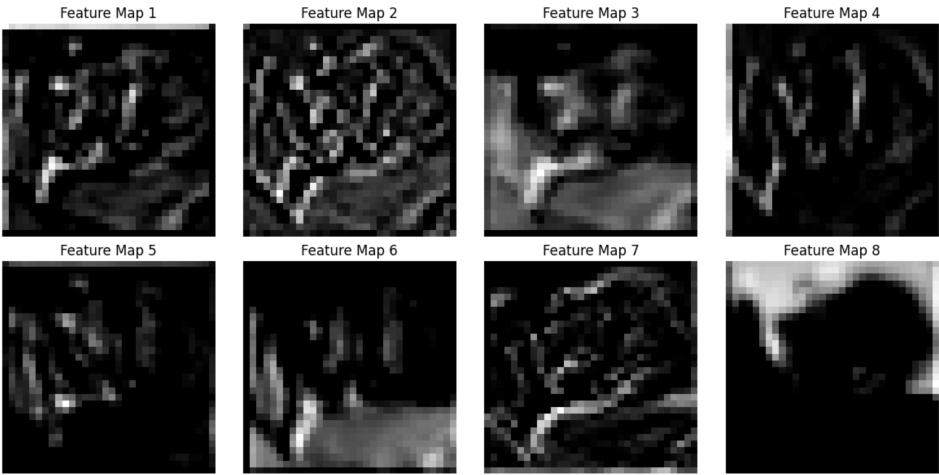


Figure 5: Feature Maps

Inference: The eight feature maps show different outputs produced by the first convolution layer for the same input image. The maps are different from each other because each filter produces a different activation pattern and learns a different set of features from the image.

9.6 Confusion Matrix

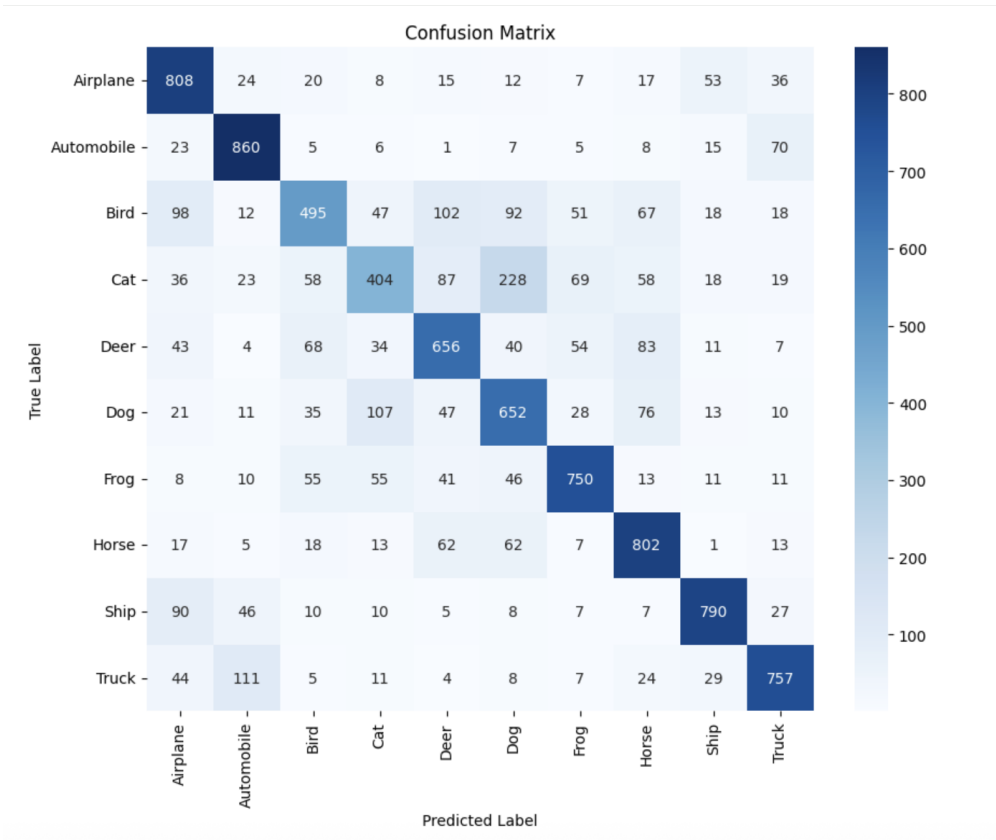


Figure 6: Confusion Matrix

Inference: The model classifies classes like Automobile, Airplane and Ship better, as they have higher values along the diagonal. Cat has the lowest recall of 0.40 and is often confused with Dog, with 228 Cat images predicted as Dog. Overall, the model performs around 70% accuracy but has more difficulty distinguishing between some similar classes.

10. Additional Exercises

10.1 Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.

Given:

$$N = 64, \quad F = 5, \quad S = 2, \quad P = 2$$

The output size is calculated using:

$$O = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1$$

Substituting the given values:

$$\begin{aligned} O &= \left\lfloor \frac{64 - 5 + 2(2)}{2} \right\rfloor + 1 \\ &= \left\lfloor \frac{63}{2} \right\rfloor + 1 \\ &= 31 + 1 \end{aligned}$$

$$\boxed{O = 32}$$

Since the input is 64×64 and the same calculation applies to both dimensions, the output feature map size is:

$$\boxed{32 \times 32}$$

Thus, using a stride of 2 reduces the spatial dimensions approximately by half, while the padding of 2 helps preserve the spatial information at the boundaries.

10.2 Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input

The input is an RGB image, so it has 3 input channels.

Given:

$$\text{Number of filters} = 64$$

$$\text{Kernel size} = 3 \times 3$$

Each filter operates across all three input channels. Therefore, the number of weights in one filter is:

$$3 \times 3 \times 3 = 27$$

One bias term is also associated with each filter:

$$27 + 1 = 28$$

For 64 filters:

$$\text{Total parameters} = (3 \times 3 \times 3 + 1) \times 64$$

$$= 28 \times 64$$

$$\boxed{1792}$$

Therefore, the convolution layer contains **1792 trainable parameters**. The calculation shows that the parameter count depends on the kernel size, number of input channels and number of filters.

10.3 ReLU and Sigmoid Activation Functions

ReLU and Sigmoid are activation functions used to add non-linearity, but they work differently. ReLU is defined as:

$$f(x) = \max(0, x)$$

It gives 0 for negative values and keeps positive values unchanged. Sigmoid is defined as:

$$f(x) = \frac{1}{1 + e^{-x}}$$

and gives outputs between 0 and 1. Sigmoid can suffer from the vanishing gradient problem for very large or small inputs, while ReLU avoids this for positive values. However, ReLU can have the dying ReLU problem when neurons keep receiving negative values. Hence, ReLU is preferred in the hidden layers of our CNN, while Sigmoid is mainly used for binary classification. For our 10-class CIFAR-10 problem, Softmax is used at the output layer.

10.4 Max Pooling and Average Pooling

Both pooling methods used a 2×2 window with stride 2. Therefore, both methods reduce the spatial dimensions by half and produce the same output shape:

$$(1, 16, 16, 3)$$

The experimental test accuracies were:

Pooling Method	Test Accuracy
Max Pooling	0.6391
Average Pooling	0.5969

Thus, Max Pooling performed approximately 4.22 percentage better than Average Pooling in this experiment. Max Pooling retains the strongest activation within each pooling window, which helps preserve prominent features detected by the convolution layer. Average Pooling instead averages all values in the window, which can weaken strong feature responses. Since the CNN relies on these feature responses for classification, retaining the strongest activations resulted in better test accuracy for Max Pooling.

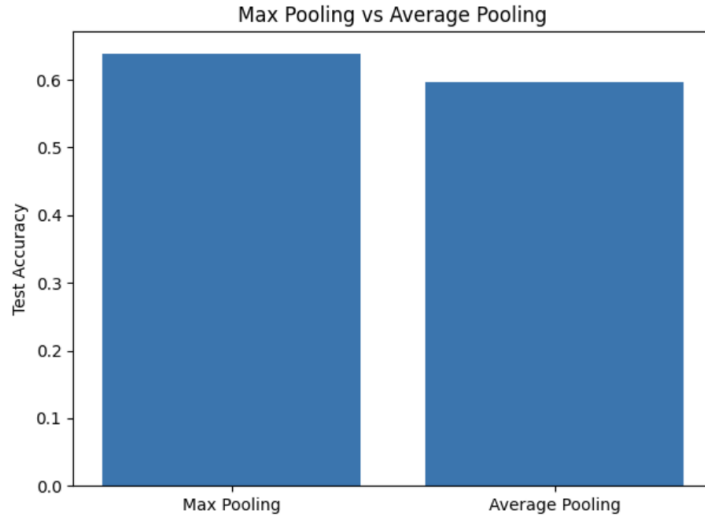


Figure 7: Test accuracy comparison between Max Pooling and Average Pooling

10.5 Effect of Number of Convolution Filters

The effect of changing the number of filters in the first convolution layer was studied using 16 and 64 filters. Both models used the same architecture apart from the number of filters and were trained for 10 epochs.

Filters	Test Accuracy	Training Time (s)	Parameters
16	0.6965	56.00	535434
64	0.6931	61.83	564426

The 64-filter model has 28,992 more parameters than the 16-filter model because four times as many feature detectors are used in the first convolution layer. This increased the training time from approximately 56.00 seconds to 61.83 seconds, since more feature maps and computations had to be processed. However, the test accuracy decreased slightly from 69.65% to 69.31%. This indicates that simply increasing the number of filters did not improve generalization in this experiment. The additional filters provide greater feature representation capacity, but the model may not benefit from this extra capacity with only 10 training epochs and without other changes to the architecture or regularization. Hence, more filters increase computational cost and model capacity, but do not necessarily guarantee higher accuracy.

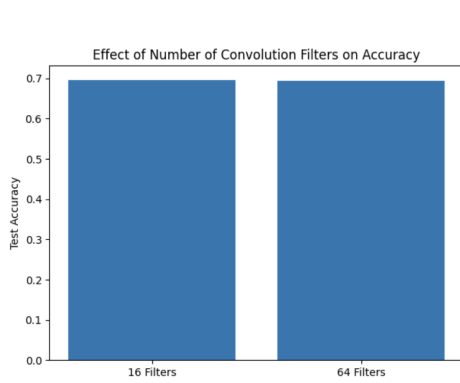


Figure 8: Test accuracy for 16 and 64 filters

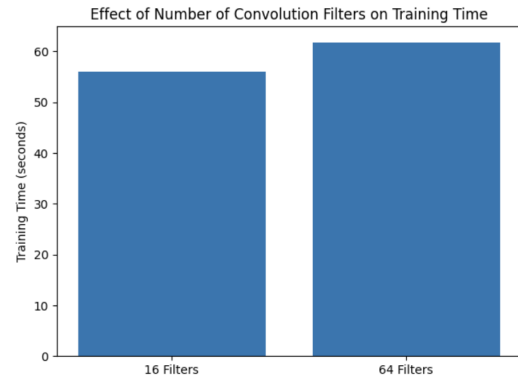


Figure 9: Training time for 16 and 64 filters

Inference: Increasing the filters from 16 to 64 increased the computational requirement and training time, but did not improve test accuracy much in this case. The result shows that increasing model capacity alone is not sufficient; the additional features must also contribute to better generalization.

11. Results

The final evaluation metrics were calculated after loading the best model weights saved during training.

Metric	Value
Training Accuracy	0.7616000175476074
Testing Accuracy	0.6973999738693237
Precision	0.6951932197964467
Recall	0.6974
F1-score	0.6922057868507405
Number of Parameters	545098

Classification Report

Class	Precision	Recall	F1-score	Support
Airplane	0.68	0.81	0.74	1000
Automobile	0.78	0.86	0.82	1000
Bird	0.64	0.49	0.56	1000
Cat	0.58	0.40	0.48	1000
Deer	0.64	0.66	0.65	1000
Dog	0.56	0.65	0.61	1000
Frog	0.76	0.75	0.76	1000
Horse	0.69	0.80	0.74	1000
Ship	0.82	0.79	0.81	1000
Truck	0.78	0.76	0.77	1000

The overall accuracy obtained on the test set was 0.6974. The best F1-score was obtained for Automobile with 0.82, followed by Ship with 0.81. Cat had the lowest F1-score of 0.48 among the ten classes.

12. Discussion

1. Why is convolution preferred over fully connected layers for images?

Convolution is preferred for images because it keeps and uses the spatial relationship between nearby pixels. Instead of connecting every pixel to every neuron, a convolution layer uses small filters on local regions of the image to detect features such as edges, textures and patterns. In our experiment, the convolution layers processed the $32 \times 32 \times 3$ CIFAR-10 images and generated feature maps that were later used for classification. The same filter is applied at different locations, so it can detect a similar feature wherever it appears in the image. This weight sharing greatly reduces the number of parameters compared with directly connecting all image pixels to a fully connected layer. Hence, convolution is a more efficient way to learn image features while keeping the spatial information.

2. How does stride affect the feature map size?

Stride determines how many pixels the convolution filter moves at each step. In our experiment, using a 3×3 kernel with same padding and stride 1 produced a feature map of size 32×32 , while stride 2 produced a feature map of size 16×16 . So, increasing the stride reduces the number of filter positions and decreases the spatial dimensions of the output. We also observed this with valid padding, where stride 1 produced a larger feature map than stride 2. A larger stride reduces computation and spatial resolution, but it can also lose some fine details. Therefore, stride provides a balance between keeping image details and reducing the size of the feature representation.

3. What is the role of padding?

Padding adds extra pixels around the boundary of an input before convolution is performed. In our experiment, same padding with a 3×3 kernel and stride 1 preserved the spatial size of the 32×32 input, producing a 32×32 feature map. In contrast, valid padding does not add extra pixels, so the same convolution reduced the spatial dimensions to 30×30 . Padding also allows information near the image boundaries to be included in the convolution. It prevents the spatial dimensions from shrinking too quickly when multiple convolution layers are used. Thus, the choice of padding affects the feature map size and the amount of boundary information retained.

4. Why is pooling used?

Pooling is used to reduce the spatial dimensions of feature maps while keeping important information about the detected features. In our experiment, a 2×2 pooling operation with stride 2 reduced the spatial dimensions from 32×32 to 16×16 . This reduces the computation needed by the following layers and gives a more compact representation of the image. We compared Max Pooling and Average Pooling and obtained test accuracies of 0.6391 and 0.5969 respectively. Max Pooling performed better because it keeps the strongest activation from each region, which can preserve important feature responses. Therefore, the experiment showed that both the pooling method and the reduction in size can affect classification performance.

5. How do feature maps represent image characteristics?

A feature map shows how strongly a particular convolution filter responds to different regions of an image. Different filters have different weights, so they respond to different visual patterns in the input. In our experiment, the first convolution layer produced 32 feature maps of size 32×32 , and eight of them were visualized. The different activation patterns show that each filter extracts a different representation from the same image. These can represent low-level features such as edges, intensity changes and textures. The feature maps are then passed through later layers, where more useful representations are learned for classification.

6. Why do CNNs require fewer parameters than MLPs?

CNNs require fewer parameters because they use local connectivity and weight sharing. A convolution filter works only on a small region of the image instead of connecting to every pixel. The same filter weights are reused at different spatial positions, so the same parameters can detect a feature at different locations. In our parameter calculation, a convolution layer with 64 filters of size 3×3 and an RGB input required only 1792 trainable parameters. This is much smaller than using separate weights for every pixel connection in a fully connected network. Therefore, CNNs can learn image features effectively while using fewer trainable parameters.

13. Source Code

The complete source code and experiment notebook are available in the following GitHub repository:

https://github.com/srinithi0406/Deep_Learning/tree/main/DL_3

14. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.